

Reviewer Pack — Free Cumulant Gap in Read-Twice BPs for 3-SAT

Entry 8818af330631 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 23:54:54 UTC

Free Cumulant Gap in Read-Twice BPs for 3-SAT

Entry ID: 8818af330631 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 23:54:54 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For a read-twice BP computing a 3-SAT instance with n variables, the maximum free cumulant of its truth table is $O(\log n)$. For the IP_2 function (inner product mod 2), which requires exponential size read-twice BPs, the maximum free cumulant is $\Omega(n)$.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative independence structures in random matrices, which may reveal hidden algebraic constraints in BP computation. The IP_2 function's exponential lower bound suggests its cumulant growth diverges from read-twice BPs' logarithmic capacity.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dde6a2257101bee5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def free_cumulant(truth_table):
        n = len(truth_table)
        cumulants = [0] * (n + 1)
        for i in range(n):
            for j in range(i, n):
                if truth_table[i][j]:
                    cumulants[j - i + 1] += 1
        return max(cumulants)

    def moment_cumulant_inversion(moments):
        cumulants = [0] * len(moments)
        for k in range(1, len(moments)):
            cumulants[k] = moments[k]
            for j in range(k - 1, 0, -1):
                cumulants[j] -= (j + 1) * cumulants[j + 1]
            cumulants[0] /= k
        return cumulants

    def generate_3sat_instance(n):
        instance = []
        for _ in range(2**n):
            assignment = [random.choice([True, False]) for _ in range(n)]
            clause = random.choice([assignment, [not x for x in assignment]])
            instance.append(clause)
        return instance

    def truth_table(instance):
        n = len(instance[0])
        table = [[False] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            assignment = [bool(i >> j & 1) for j in range(n)]
            for clause in instance:
                if all([assignment[j] or not x for j, x in enumerate(clause)]):
                    table[i][i ^ sum(1 << j for j, x in enumerate(assignment))] = True
        return table
```

```

def read_twice_bp(truth_table):
    n = len(truth_table)
    bp = [[False] * (2**n) for _ in range(2**n)]
    for i in range(n):
        for j in range(2**n):
            if truth_table[i][j]:
                bp[j][j ^ 1 << i] = True
    return bp

def cumulants(bp):
    n = len(bp)
    moments = [0] * (n + 1)
    for i in range(n):
        for j in range(i, n):
            if bp[i][j]:
                moments[j - i + 1] += 1
    return moment_cumulant_inversion(moments)

def ip2_function():
    bp = [[False] * (2**40) for _ in range(2**40)]
    for i in range(2**40):
        for j in range(i, 2**40):
            if i & j == 0:
                bp[i][j] = True
    return bp

n = 40
instance = generate_3sat_instance(n)
truth_table_3sat = truth_table(instance)
cumulant_3sat = free_cumulant(truth_table_3sat)

ip2_bp = ip2_function()
cumulants_ip2 = cumulants(ip2_bp)

max_cumulant_3sat = max(cumulant_3sat)
max_cumulant_ip2 = max(cumulants_ip2)

conjecture_holds = max_cumulant_3sat <= 5 * math.log(n) and max_cumulant_ip2 > 40
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "max_free_cumulant",
    "metric_value": max_cumulant_3sat,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support/falsification criteria. | next: Run test with extended timeout and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	41438
2	preregistration	ollama_remote	qwen3:8b	0	0	24864
3	novelty	ollama_remote	qwen3:8b	0	0	20706
4	novelty	ollama_remote	qwen3:8b	0	0	15158
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14331
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12498
7	judge	ollama_remote	qwen3:8b	0	0	55272

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 184267 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/8818af330631.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8818af330631.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8818af330631.tar.gz` (if generated)